DSM — Dependency Suitability Module

DSM — Dependency Suitability Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Dependency Standard (SDS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Dependency Suitability Governance

Module ID: SLGA-SDS-DSM-003

Origin ID: OOF.CM.SLGA.03.03.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Dependency Suitability Module (DSM) governs the determination,

justification, validation, preservation, and reassessment of software dependency suitability within a defined software purpose, requirements baseline, technical environment, operational context, and intended lifecycle conditions.

DSM establishes the methodological conditions under which an identified

software dependency may be recognized as necessary, appropriate, compatible, sufficiently constrained, and suitable for a specific software reliance relationship.

It governs the relationship between dependency purpose, functional

contribution, demonstrated necessity, alternative dependency options, technical compatibility, environmental conditions, operational constraints, evidence, uncertainty, and the requirements of the dependent software system.

DSM distinguishes dependency availability from suitability, functional

capability from justified necessity, successful installation from demonstrated compatibility, and current operational success from suitability across the intended software context.

It applies to internal and external software components, libraries,

packages, frameworks, plugins, runtime platforms, software services, build resources, generated components, embedded dependencies, and material direct or transitive dependency relationships.

DSM applies regardless of whether the dependency was selected by a human

developer, an automated development tool, an AI coding assistant, or an autonomous software development agent.

Its principal output is:

Governed Dependency Suitability State

Its foundational distinction is:

Dependency Functionality ≠ Governed Dependency Suitability.

Operational Role

The operational role of DSM is:

To ensure that software dependencies are assessed against the actual requirements, purpose, constraints, technical conditions, and intended operational context of the software system before their suitability is recognized or relied upon in subsequent dependency governance.

DSM establishes the suitability-governance foundation of SDS.

It receives sufficiently identified components and their relevant origin

and traceability information.

It determines whether the dependency is justified for the specific software context in which it is intended to operate.

DSM does not independently authorize dependency introduction, certify software security, establish legal compliance, or guarantee dependency continuity.

It provides a structured suitability determination for competent downstream

governance.

Operational Question

Under which evidenced requirements, technical conditions, compatibility

boundaries, and operational constraints is a particular software dependency necessary or appropriate for the software system that relies upon it?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

DSM operates within:

Software Dependency Standard (SDS)

The SDS internal governance flow is:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

DSM governs:

Dependency Suitability

It is the third Core Module of SDS.

Its principal upstream inputs are:

Governed Software Component Identification State

from:

CIM — Component Identification Module

and:

Governed Dependency Origin & Traceability State

from:

DOTM — Dependency Origin & Traceability Module

DSM may also receive relevant requirements references from:

Software Requirements Standard (SRS)

and relevant creation-context information from:

Software Creation Standard (SCS)

Its principal output is:

Governed Dependency Suitability State

This output supports:

CICM — Component Introduction & Change Module

and:

DCRM — Dependency Continuity & Risk Module

Suitability may require reassessment throughout the software lifecycle.

• Governed Space

Software Dependency Suitability Governance

This space governs the conditions under which a software dependency is considered appropriate for a defined software reliance context.

Its principal dimensions include:

• Dependency suitability governance objects

• Dependent software purpose

• Governed requirements alignment

• Dependency purpose

• Dependency necessity

• Functional appropriateness

• Technical compatibility

• Version compatibility

• Interface compatibility

• Runtime compatibility

• Platform compatibility

• Environment compatibility

• Dependency interaction compatibility

• Performance suitability

• Resource suitability

• Operational constraints

• Dependency assumptions

• Alternative dependency options

• Substitutability references

• Suitability evidence

• Evidence competence

• Suitability validation

• Suitability uncertainty

• Suitability conflicts

• Suitability gaps

• Conditional suitability

• Suitability drift

• Suitability continuity

• Suitability governance states

This governed space is technology-independent.

Dependency Suitability Governance Object

Every DSM assessment shall establish a Dependency Suitability Governance

Object.

The object represents the suitability conditions of a specific dependency

relationship.

It should identify:

Governance Object Identity: The identifiable suitability assessment.

Dependent Software Object: The software system relying on the dependency.

Dependency Component: The component identified through CIM.

Dependency Origin Reference: Relevant origin and traceability information

from DOTM.

Dependency Relationship: The material reliance relationship.

Software Purpose: The relevant governed software purpose.

Requirements Reference: Applicable software requirements.

Dependency Purpose: The function the dependency is expected to perform.

Necessity Basis: The justification for the dependency.

Compatibility Conditions: Relevant technical and operational

compatibility requirements.

Operational Constraints: Conditions limiting suitable use.

Suitability Evidence: Evidence supporting the assessment.

Suitability Uncertainty: Material unresolved suitability conditions.

Suitability Governance State: The current recognized suitability

condition.

The suitability governance object shall remain distinguishable from the

dependency component itself.

Dependency Component Identity ≠ Dependency Suitability Governance Object.

Dependent Software Purpose Alignment

DSM shall identify the governed purpose of the dependent software system.

Suitability shall be evaluated in relation to that purpose.

A dependency may be technically capable of performing a function while contributing nothing necessary or justified to the intended software purpose.

Relevant assessment questions include:

• What purpose does the software system serve?

• What function is the dependency expected to support?

• Is that function relevant to the governed purpose?

• Does the dependency introduce functionality outside the intended scope?

• Does the dependency conflict with established purpose constraints?

Dependency Capability ≠ Software Purpose Alignment.

Requirements-Based Suitability

DSM shall evaluate dependency suitability against applicable governed

software requirements.

Relevant requirements may concern:

• Functional behavior

• Performance

• Reliability

• Interoperability

• Availability

• Maintainability

• Technical constraints

• Operational conditions

• Data handling

• Environmental requirements

• Lifecycle expectations

The presence of a dependency shall not be treated as evidence that it satisfies the relevant requirements.

Dependency Inclusion ≠ Requirements Satisfaction.

Dependency Purpose Identification

DSM shall establish why the dependency is being considered or used.

The dependency purpose should identify:

• The function it supports

• The requirements it addresses

• The software component that relies upon it

• The expected contribution

• The relevant operating conditions

A dependency purpose shall be sufficiently specific to support meaningful

suitability assessment.

General Dependency Utility ≠ Defined Dependency Purpose.

Dependency Necessity

DSM shall assess whether the dependency is necessary or sufficiently justified for the intended software context.

Necessity may arise because:

• The dependency provides required functionality

• The dependency enables an essential integration

• The dependency supports an established technical constraint

• The dependency provides a justified implementation mechanism

• The dependency is required by another accepted dependency

• The dependency supports a necessary development or execution function

Necessity shall not be inferred solely from convenience, popularity, or

developer preference.

Dependency Convenience ≠ Dependency Necessity.

Dependency Necessity Evidence

DSM shall preserve evidence supporting dependency necessity.

Relevant evidence may include:

• Requirements relationships

• Architecture decisions

• Functional analysis

• Technical constraints

• Comparative assessments

• Dependency relationship records

• Demonstrated implementation needs

A necessity claim shall remain distinguishable from a demonstrated

necessity condition.

Claimed Dependency Necessity ≠ Evidenced Dependency Necessity.

Functional Suitability

DSM shall assess whether the dependency provides the functionality required

by the dependent software.

The assessment should consider:

• Supported functions

• Required behavior

• Functional limitations

• Relevant interfaces

• Input and output conditions

• Material functional assumptions

A dependency may provide the requested function while introducing additional behavior that is unsuitable for the intended context.

Functional Availability ≠ Functional Suitability.

Functional Scope Alignment

DSM shall assess whether the dependency's functional scope is appropriate for the dependent software.

Relevant concerns may include:

• Unnecessary capabilities

• Excessive functionality

• Missing required functions

• Uncontrolled feature dependencies

• Material behavior outside the intended software scope

A large dependency may be justified where its capabilities are necessary.

A small dependency may be unsuitable where it introduces incompatible behavior.

Dependency Feature Quantity ≠ Dependency Suitability.

Technical Compatibility

DSM shall assess whether the dependency is technically compatible with the relevant software system.

Compatibility may concern:

• Programming language

• Runtime

• Operating system

• Hardware architecture

• Software interfaces

• Data formats

• Protocols

• Build processes

• Execution conditions

Compatibility shall be established relative to the intended environment.

Technical Installability ≠ Demonstrated Technical Compatibility.

Version Compatibility

DSM shall assess compatibility between the dependency version and the

relevant software environment.

Relevant conditions may include:

• Supported version ranges

• Required runtime versions

• Interface changes

• Deprecated functions

• Version-specific behavior

• Relevant migration conditions

A dependency's newest available version shall not automatically be

considered the most suitable version.

Latest Dependency Version ≠ Most Suitable Dependency Version.

Interface Compatibility

DSM shall assess whether the dependency's interfaces are compatible with the dependent software.

Relevant considerations may include:

• API contracts

• Data structures

• Function signatures

• Communication protocols

• Error behavior

• Compatibility guarantees

• Interface assumptions

The existence of a callable interface does not independently establish that its behavior is suitable.

Interface Accessibility ≠ Interface Compatibility.

Runtime Compatibility

DSM shall assess whether the dependency can operate under the intended runtime conditions.

Relevant conditions may include:

• Runtime versions

• Execution environments

• Dynamic loading behavior

• Runtime resource requirements

• Execution privileges

• Runtime configuration assumptions

A successful development-environment execution shall not automatically

establish production runtime compatibility.

Development Execution Success ≠ Production Runtime Compatibility.

Platform Compatibility

DSM shall assess compatibility with material platform dependencies.

Relevant platforms may include:

• Operating systems

• Database platforms

• Container environments

• Application servers

• Managed runtime services

• Cloud execution platforms

Compatibility shall reflect the intended platform conditions.

Platform Support Claim ≠ Verified Platform Compatibility.

Build and Development Compatibility

DSM shall assess whether dependencies required during software creation are compatible with the intended development and build processes.

Relevant conditions may include:

• Compiler requirements

• Build tool compatibility

• Code generation dependencies

• Package resolution behavior

• Testing environment requirements

• Reproducibility constraints

A dependency may be suitable for runtime operation while being incompatible with the established build environment.

Runtime Suitability ≠ Build Suitability.

Dependency Interaction Compatibility

DSM shall assess material compatibility relationships between dependencies.

A dependency may function correctly in isolation while interacting incompatibly with other components.

Relevant conditions may include:

• Conflicting library versions

• Shared runtime resources

• Incompatible protocols

• Conflicting interfaces

• Resource contention

• Material configuration interactions

Individual Component Compatibility ≠ Dependency Network Compatibility.

Environment-Specific Suitability

DSM shall distinguish suitability across different software environments.

Relevant environments may include:

• Development

• Testing

• Staging

• Production

• Maintenance

• Recovery environments

A dependency may be suitable for one environment but unsuitable for another.

Environment-Specific Suitability ≠ Universal Suitability.

Performance Suitability

DSM shall assess whether the dependency's relevant performance

characteristics are appropriate for the intended software context.

Relevant dimensions may include:

• Execution latency

• Throughput

• Memory consumption

• Processing overhead

• Resource contention

• Response consistency

• Scalability constraints

Performance suitability shall be assessed against actual requirements and

material operating conditions.

Benchmark Performance ≠ Contextual Performance Suitability.

Resource Suitability

DSM shall assess whether the dependency's resource requirements are compatible with the intended software environment.

Relevant resources may include:

• Computational capacity

• Memory

• Storage

• Network capacity

• Energy

• Infrastructure dependencies

• External service capacity

Detailed resource efficiency and value optimization remain within competent governance domains.

Resource Availability ≠ Justified Resource Consumption.

Dependency Constraints

DSM shall identify material constraints affecting dependency suitability.

Constraints may arise from:

• Technical architecture

• Operating environment

• Software requirements

• Interface obligations

• Deployment conditions

• Maintenance expectations

• Resource limitations

• Applicable organizational requirements

A dependency may be suitable only within defined constraints.

Conditional Suitability ≠ Unrestricted Suitability.

Dependency Assumptions

DSM shall identify material assumptions underlying suitability claims.

Assumptions may concern:

• Expected runtime behavior

• Interface stability

• Version compatibility

• Resource availability

• Service behavior

• Platform support

• Configuration conditions

• Required operating context

Unsupported assumptions shall not be represented as verified suitability

conditions.

Assumed Compatibility ≠ Demonstrated Compatibility.

Alternative Dependency Assessment

DSM shall support assessment of relevant alternatives where alternatives materially affect the suitability determination.

An alternative may be:

• Another library

• Another framework

• An internal component

• A different service

• A different implementation approach

• Removal of unnecessary dependency functionality

DSM does not require exhaustive evaluation of every available alternative.

It requires that material alternative considerations be appropriately recognized.

Available Alternative ≠ Suitable Alternative.

Dependency Selection Justification

DSM shall establish whether the suitability of a selected dependency is sufficiently justified.

Relevant considerations may include:

• Requirements alignment

• Functional appropriateness

• Compatibility

• Constraints

• Relevant alternatives

• Supporting evidence

• Material uncertainty

Selection justification shall remain distinct from authorization to

introduce the dependency.

Justified Dependency Selection ≠ Authorized Dependency Introduction.

Dependency Suitability and Provenance

DSM shall consider relevant provenance limitations received from DOTM.

An unresolved origin may affect the evidence available for a suitability assessment.

However, provenance and suitability remain distinct governance dimensions.

A dependency may have verified provenance but be technically unsuitable.

A dependency may appear functionally suitable while its origin remains unresolved.

Verified Dependency Origin ≠ Demonstrated Dependency Suitability.

Dependency Suitability and Security

DSM may recognize security-related requirements and constraints relevant to suitability.

It shall not independently replace competent security assessment.

Detailed software security governance remains within:

Software Security Standard (SSS)

A dependency may be functionally suitable while presenting unacceptable security conditions.

Functional Suitability ≠ Security Acceptance.

Dependency Suitability and Legal Constraints

DSM shall preserve material legal, contractual, or licensing constraints

where they affect the permitted software context.

It shall not independently determine legal compliance or interpret disputed

legal obligations.

Competent legal assessment remains necessary where applicable.

Technical Suitability ≠ Legal Permission.

Dependency Suitability and Authority

DSM establishes suitability conditions.

It does not independently authorize dependency introduction.

Detailed introduction and change governance remains within:

CICM — Component Introduction & Change Module

A dependency may be technically suitable but introduced without legitimate authority.

Dependency Suitability ≠ Dependency Introduction Authority.

Dependency Suitability and Continuity

DSM may identify maintenance, availability, or support assumptions that materially affect suitability.

It does not independently determine long-term dependency continuity.

Detailed continuity and risk governance remains within:

DCRM — Dependency Continuity & Risk Module

Present Suitability ≠ Dependency Continuity Assurance.

Autonomous Dependency Selection

DSM shall support suitability assessment of dependencies selected by

autonomous software development systems.

An autonomous agent may select a component based on:

• Functional matching

• Search ranking

• Popularity

• Installation convenience

• Generated recommendations

• Automated compatibility predictions

These selection mechanisms shall not automatically establish governed

suitability.

Autonomous Dependency Selection ≠ Governed Dependency Suitability.

AI-Generated Suitability Claims

AI systems may produce dependency recommendations and suitability explanations.

DSM shall distinguish such explanations from evidence-supported suitability

assessments.

An AI-generated claim may be useful as an assessment input.

It shall not independently establish compatibility, necessity, or

appropriateness.

AI-Generated Suitability Explanation ≠ Verified Suitability Evidence.

Dependency Suitability Evidence

DSM shall preserve evidence supporting material suitability claims.

Evidence may include:

• Requirements references

• Compatibility documentation

• Test results

• Technical evaluations

• Architecture records

• Performance measurements

• Interface specifications

• Environment assessments

• Comparative analyses

• Operational observations

Evidence shall remain associated with the specific suitability claim it

supports.

Suitability Evidence Presence ≠ Suitability Claim Validity.

Suitability Evidence Competence

DSM shall assess whether available evidence is competent to support the relevant suitability determination.

For example:

An installation record may support installation success.

A functional test may support behavior under tested conditions.

A compatibility assessment may support specific environment compatibility.

A performance benchmark may support measured performance under defined conditions.

These records do not establish identical suitability facts.

Evidence of One Suitability Dimension ≠ Evidence of Complete Suitability.

Suitability Validation

DSM shall determine whether material suitability conditions are sufficiently supported.

Validation should consider:

• Software purpose

• Requirements alignment

• Dependency necessity

• Functional appropriateness

• Technical compatibility

• Environmental compatibility

• Dependency interactions

• Material constraints

• Evidence competence

• Material uncertainty

Validation shall remain limited to the conditions actually assessed.

Validated Suitability Scope ≠ Universal Dependency Suitability.

Suitability Evidence Integrity

DSM shall preserve the integrity of material suitability evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Assessment context

• Temporal relevance

• Version correspondence

• Test environment correspondence

• Material contradictions

• Evidence completeness

Evidence integrity does not independently establish suitability.

Intact Test Evidence ≠ Complete Suitability Assurance.

Dependency Suitability Uncertainty

Suitability uncertainty may arise where:

• Requirements are incomplete

• Compatibility is insufficiently tested

• The relevant operating environment is unknown

• Performance behavior is uncertain

• Dependency interactions remain unresolved

• Material constraints are unidentified

• Alternative assessments are incomplete

• Suitability evidence is insufficient

DSM shall preserve material uncertainty.

Uncertain Suitability ≠ Demonstrated Unsuitability.

Dependency Suitability Conflict

A suitability conflict exists where material evidence supports incompatible suitability conclusions.

Examples may include:

• Contradictory compatibility assessments

• Conflicting performance measurements

• Inconsistent environmental requirements

• Incompatible dependency version claims

• Conflicting functional evaluations

DSM shall preserve the conflict and its supporting evidence.

Conflicting Suitability Evidence ≠ Permission to Select the Preferred

Conclusion.

Dependency Suitability Gap

A Dependency Suitability Gap exists where a material suitability condition has not been sufficiently established.

Examples may include:

• Missing compatibility evidence

• Unjustified dependency necessity

• Unresolved runtime constraints

• Insufficient requirements alignment

• Missing dependency interaction assessment

• Unverified performance assumptions

DSM shall identify the gap and its material significance.

Suitability Gap ≠ Proven Dependency Failure.

Critical Suitability Gap

A Critical Suitability Gap exists where insufficient suitability evidence prevents competent assessment of a materially consequential dependency.

Examples may include:

- An unverified runtime dependency supporting a critical software

function. - A component whose compatibility with the intended operating environment is unresolved. - A dependency whose required performance cannot be established. - An autonomous agent-selected component with unresolved requirements alignment.

DSM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software rejection, suspension, or legal liability.

Critical Suitability Gap ≠ Automatic Software Rejection.

Conditional Dependency Suitability

A dependency may be suitable only under specific conditions.

Conditions may include:

• Defined software versions

• Particular runtime environments

• Required configurations

• Resource limits

• Interface constraints

• Restricted functional use

• Additional verification requirements

DSM shall preserve the conditions and their supporting evidence.

Conditionally Suitable ≠ Unconditionally Suitable.

Dependency Suitability Drift

Dependency Suitability Drift occurs when the actual software context materially diverges from the conditions under which suitability was established.

Possible causes include:

• Software requirements changes

• Dependency updates

• Runtime changes

• Platform migrations

• Interface changes

• Configuration changes

• Increased operational demand

• Changed dependency interactions

DSM shall identify material drift and support reassessment.

Previously Suitable Dependency ≠ Permanently Suitable Dependency.

Dependency Suitability Change

DSM shall preserve material changes to suitability assessments.

Changes may include:

• Revised requirements alignment

• New compatibility evidence

• Updated dependency versions

• Changed operational constraints

• Resolved suitability gaps

• Newly identified conflicts

• Revised suitability conclusions

Historical suitability states shall remain reconstructable where material.

Suitability Reassessment ≠ Historical Suitability Erasure.

Dependency Suitability Continuity

DSM shall preserve the relationship between a dependency's recognized

suitability and the conditions under which that recognition remains applicable.

Continuity may involve:

• Software version changes

• Dependency updates

• Environment changes

• Requirements revisions

• Architecture changes

• Service interface changes

• Software modernization

Suitability continuity concerns the continued applicability of suitability

determinations.

It does not independently establish supplier support or operational availability.

Suitability Continuity ≠ Operational Dependency Continuity.

Dependency Suitability Governance States

DSM shall support identifiable suitability governance states.

Possible states include:

Under Assessment: Suitability conditions are being evaluated.

Suitable: Material suitability conditions are sufficiently established within the assessed scope.

Conditionally Suitable: Suitability depends on explicit conditions.

Partially Suitable: Some material suitability dimensions remain

unresolved or unsupported.

Evidence-Limited: Available evidence is insufficient for the required determination.

Necessity-Unresolved: Dependency necessity is not sufficiently established.

Compatibility-Unresolved: Material compatibility conditions remain

uncertain.

Constraint-Unresolved: Material suitability constraints remain

unidentified or unresolved.

Conflicted: Material suitability evidence is contradictory.

Unsuitable: The dependency does not satisfy material suitability conditions within the assessed scope.

Suitability-at-Risk: Material changes threaten continued suitability.

Superseded: A later suitability state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not be interpreted as dependency introduction

authorization or software product acceptance.

Dependency Suitable ≠ Dependency Authorized.

Dependency Suitability Recognition

DSM shall determine whether a dependency is sufficiently suitable for the defined software context.

Recognition should consider:

• Dependency identity

• Dependency purpose

• Requirements alignment

• Necessity

• Functional appropriateness

• Technical compatibility

• Environmental compatibility

• Interaction compatibility

• Performance suitability

• Resource suitability

• Material constraints

• Alternative considerations

• Supporting evidence

• Material uncertainty

• Suitability conflicts

• Suitability continuity

Recognition shall remain limited to the assessed context.

Recognized Suitability ≠ Complete Dependency Governance.

Dependency Suitability Reassessment

DSM shall support reassessment when material suitability conditions change.

Triggers may include:

• New software requirements

• Dependency version changes

• Runtime changes

• Platform changes

• Interface changes

• Configuration changes

• New performance evidence

• Resource constraints

• Dependency interaction changes

• New operational observations

• Suitability conflicts

• Suitability drift

• Material software lifecycle changes

Reassessment shall preserve material historical suitability records.

Governed Dependency Suitability State

The principal output of DSM is:

Governed Dependency Suitability State

The state shall preserve sufficient information to identify:

Suitability Governance Object: The dependency suitability assessment.

Dependent Software Object: The software system relying on the dependency.

Component Identity: The component identified by CIM.

Dependency Origin Reference: Relevant DOTM provenance information.

Dependency Relationship: The material software reliance relationship.

Software Purpose Reference: The relevant governed software purpose.

Requirements Reference: Applicable requirements.

Dependency Purpose: The function the dependency is expected to perform.

Necessity Assessment: The justification for dependency reliance.

Functional Suitability: The relevant functional appropriateness.

Technical Compatibility: The established technical compatibility

conditions.

Version Compatibility: The relevant version relationships.

Interface Compatibility: The relevant interface conditions.

Runtime Compatibility: The applicable runtime conditions.

Platform Compatibility: The relevant platform conditions.

Environment Suitability: The assessed operating contexts.

Dependency Interaction Compatibility: Material interactions with other

dependencies.

Performance Suitability: Relevant performance conditions.

Resource Suitability: Relevant resource requirements.

Dependency Constraints: Material limits on suitable use.

Dependency Assumptions: Material assumptions underlying suitability.

Alternative Assessment: Relevant alternative dependency considerations.

Suitability Evidence: Evidence supporting the assessment.

Evidence Competence: The suitability of evidence for the assessed claim.

Evidence Integrity: The condition of supporting evidence.

Suitability Validation: The scope and result of suitability assessment.

Suitability Uncertainty: Material unresolved conditions.

Suitability Conflicts: Contradictory suitability evidence.

Suitability Gaps: Missing material suitability conditions.

Critical Suitability Gaps: Material gaps requiring competent attention.

Conditional Suitability: Explicit conditions governing suitability.

Suitability Drift: Divergence from previously recognized conditions.

Suitability Continuity: Continued applicability of suitability

determinations.

Suitability Governance State: The recognized suitability condition.

Downstream References: Information required by competent SDS governance

modules.

The state shall not independently establish dependency introduction

authority, security acceptance, legal compliance, operational continuity, or software product acceptance.

Minimum Implementation Framework

A conforming implementation of DSM must provide the following six functional capabilities.

• Dependency Purpose, Requirements, and Necessity Governance

Capability to establish dependency suitability governance objects, identify

relevant software purposes and requirements, define dependency purpose, and assess dependency necessity.

The implementation must distinguish useful functionality from justified reliance.

• Functional, Technical, and Environmental Compatibility Governance

Capability to assess functional suitability, technical compatibility,

version compatibility, interface compatibility, runtime compatibility, platform compatibility, and environment-specific suitability.

The implementation must distinguish installation success from demonstrated compatibility.

• Dependency Interaction, Resource, Constraint, and Alternative Governance

Capability to assess material dependency interactions, performance

conditions, resource requirements, operating constraints, assumptions, and relevant alternatives.

The implementation must preserve material limitations and conditional suitability.

• Suitability Evidence, Competence, Integrity, and Validation Governance

Capability to associate suitability claims with competent evidence,

preserve evidence integrity, and determine which suitability conditions are sufficiently supported.

The implementation must distinguish evidence of isolated technical success from evidence of contextual suitability.

• Suitability Gap, Conflict, Drift, and Continuity Governance

Capability to identify material suitability gaps, critical gaps,

conflicting assessments, changing operating conditions, and suitability drift.

The implementation must preserve historical suitability states and support reassessment.

• Governed Dependency Suitability State and SDS Handoff Governance

Capability to produce a Governed Dependency Suitability State containing

sufficient purpose, requirements, necessity, compatibility, constraint, evidence, uncertainty, and continuity information for competent downstream SDS governance.

The implementation must not convert suitability into dependency introduction authorization.

GIE™ Compatibility

DSM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Dependency purpose identification

• Requirements-to-dependency mapping

• Dependency necessity analysis

• Functional suitability assessment

• Technical compatibility analysis

• Version compatibility analysis

• Interface compatibility analysis

• Runtime compatibility analysis

• Platform compatibility analysis

• Environment-specific suitability comparison

• Dependency interaction analysis

• Performance suitability assessment

• Resource suitability assessment

• Dependency constraint recognition

• Dependency assumption identification

• Alternative dependency comparison

• Suitability evidence association

• Evidence competence assessment

• Suitability validation support

• Suitability uncertainty recognition

• Suitability conflict detection

• Suitability gap detection

• Critical suitability gap detection

• Conditional suitability representation

• Suitability drift detection

• Suitability continuity assessment

• Governed suitability state representation

GIE™ may propose candidate suitability conclusions or identify possible incompatibilities.

Such outputs shall remain distinguishable from sufficiently evidenced

suitability determinations.

GIE™ must not fabricate compatibility results, invent requirements alignment, infer suitability from popularity, or convert installation success into validated suitability.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DSM governance logic is:

Receive Governed Software Component Identification State

↓

Receive Governed Dependency Origin & Traceability State

↓

Identify Dependency Suitability Governance Object

↓

Identify Dependent Software Object

↓

Identify Dependency Component and Relationship

↓

Identify Governed Software Purpose

↓

Identify Applicable Software Requirements

↓

Establish Dependency Purpose

↓

Assess Dependency Necessity

↓

Assess Functional Suitability

↓

Assess Technical Compatibility

↓

Assess Version and Interface Compatibility

↓

Assess Runtime and Platform Compatibility

↓

Assess Environment-Specific Suitability

↓

Assess Dependency Interaction Compatibility

↓

Assess Performance and Resource Suitability

↓

Identify Material Dependency Constraints

↓

Identify Dependency Assumptions

↓

Assess Relevant Dependency Alternatives

↓

Establish Dependency Selection Justification

↓

Associate Suitability Evidence

↓

Assess Evidence Competence and Integrity

↓

Validate Material Suitability Conditions

↓

Identify Suitability Uncertainty

↓

Identify Suitability Conflicts

↓

Identify Suitability Gaps and Critical Gaps

↓

Identify Conditional Suitability

↓

Identify Suitability Drift

↓

Determine Dependency Suitability Governance State

↓

Establish Governed Dependency Suitability State

↓

Preserve Suitability Continuity

↓

Handoff to CICM and Competent SDS Governance

↓

Reassess When Material Suitability Conditions Change

The principal output is:

Governed Dependency Suitability State

Governance Outputs

DSM may produce:

Dependency Suitability Governance Object

Dependent Software Purpose Reference

Requirements Alignment Record

Dependency Purpose Record

Dependency Necessity Assessment

Dependency Necessity Evidence Record

Functional Suitability Assessment

Functional Scope Alignment Record

Technical Compatibility Assessment

Version Compatibility Assessment

Interface Compatibility Assessment

Runtime Compatibility Assessment

Platform Compatibility Assessment

Build and Development Compatibility Assessment

Dependency Interaction Compatibility Assessment

Environment-Specific Suitability Record

Performance Suitability Assessment

Resource Suitability Assessment

Dependency Constraint Record

Dependency Assumption Record

Alternative Dependency Assessment

Dependency Selection Justification

Dependency Provenance Suitability Reference

Security Constraint Reference

Legal Constraint Reference

Introduction Authority Reference

Continuity Reference

Autonomous Dependency Selection Assessment

AI-Generated Suitability Claim Record

Dependency Suitability Evidence Record

Suitability Evidence Competence Assessment

Suitability Evidence Integrity Assessment

Suitability Validation Record

Suitability Uncertainty State

Suitability Conflict Record

Dependency Suitability Gap

Critical Suitability Gap

Conditional Suitability Record

Dependency Suitability Drift Record

Dependency Suitability Change Record

Dependency Suitability Continuity Record

Dependency Suitability Governance State

Dependency Suitability Recognition Record

Dependency Suitability Reassessment Record

SDS Suitability Handoff Record

The principal output is:

Governed Dependency Suitability State

Critical Distinctions

DSM preserves the following distinctions:

Dependency Functionality ≠ Governed Dependency Suitability

Dependency Component Identity ≠ Dependency Suitability Governance Object

Dependency Capability ≠ Software Purpose Alignment

Dependency Inclusion ≠ Requirements Satisfaction

General Dependency Utility ≠ Defined Dependency Purpose

Dependency Convenience ≠ Dependency Necessity

Claimed Dependency Necessity ≠ Evidenced Dependency Necessity

Functional Availability ≠ Functional Suitability

Dependency Feature Quantity ≠ Dependency Suitability

Technical Installability ≠ Demonstrated Technical Compatibility

Latest Dependency Version ≠ Most Suitable Dependency Version

Interface Accessibility ≠ Interface Compatibility

Development Execution Success ≠ Production Runtime Compatibility

Platform Support Claim ≠ Verified Platform Compatibility

Runtime Suitability ≠ Build Suitability

Individual Component Compatibility ≠ Dependency Network Compatibility

Environment-Specific Suitability ≠ Universal Suitability

Benchmark Performance ≠ Contextual Performance Suitability

Resource Availability ≠ Justified Resource Consumption

Conditional Suitability ≠ Unrestricted Suitability

Assumed Compatibility ≠ Demonstrated Compatibility

Available Alternative ≠ Suitable Alternative

Justified Dependency Selection ≠ Authorized Dependency Introduction

Verified Dependency Origin ≠ Demonstrated Dependency Suitability

Functional Suitability ≠ Security Acceptance

Technical Suitability ≠ Legal Permission

Dependency Suitability ≠ Dependency Introduction Authority

Present Suitability ≠ Dependency Continuity Assurance

Autonomous Dependency Selection ≠ Governed Dependency Suitability

AI-Generated Suitability Explanation ≠ Verified Suitability Evidence

Suitability Evidence Presence ≠ Suitability Claim Validity

Evidence of One Suitability Dimension ≠ Evidence of Complete Suitability

Validated Suitability Scope ≠ Universal Dependency Suitability

Intact Test Evidence ≠ Complete Suitability Assurance

Uncertain Suitability ≠ Demonstrated Unsuitability

Conflicting Suitability Evidence ≠ Permission to Select the Preferred

Conclusion

Suitability Gap ≠ Proven Dependency Failure

Critical Suitability Gap ≠ Automatic Software Rejection

Conditionally Suitable ≠ Unconditionally Suitable

Previously Suitable Dependency ≠ Permanently Suitable Dependency

Suitability Reassessment ≠ Historical Suitability Erasure

Suitability Continuity ≠ Operational Dependency Continuity

Dependency Suitable ≠ Dependency Authorized

Recognized Suitability ≠ Complete Dependency Governance

Cross-Architecture Boundary

DSM governs dependency suitability within SDS and SLGA™.

It does not independently establish dependency origin, introduction authority, security certification, software quality, operational continuity, or legal compliance.

Within SDS:

CIM — Component Identification Module governs component identification.

DOTM — Dependency Origin & Traceability Module governs origin,

provenance, and traceability.

DSM — Dependency Suitability Module governs necessity, appropriateness,

compatibility, constraints, and suitability.

CICM — Component Introduction & Change Module governs introduction

authority, inclusion, modification, replacement, and removal.

DCRM — Dependency Continuity & Risk Module governs criticality, risks,

maintenance, availability, substitutability, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs software requirements.

Software Creation Standard (SCS) governs software creation.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority governance.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

VEGA™ — Value Execution Governance Architecture: Relevant resource

efficiency and value governance.

AGA™ — Accountability Governance Architecture: Relevant accountability

relationships.

DSM preserves suitability relationships without replacing the competent

governance responsibilities of these domains.

Canonical Boundary

DSM begins when a software dependency has been sufficiently identified to support assessment of its necessity, appropriateness, compatibility, and suitability for a defined software context.

DSM governs:

• Dependency suitability governance objects

• Software purpose alignment

• Requirements alignment

• Dependency purpose

• Dependency necessity

• Necessity evidence

• Functional suitability

• Functional scope alignment

• Technical compatibility

• Version compatibility

• Interface compatibility

• Runtime compatibility

• Platform compatibility

• Build and development compatibility

• Dependency interaction compatibility

• Environment-specific suitability

• Performance suitability

• Resource suitability

• Dependency constraints

• Dependency assumptions

• Relevant alternatives

• Dependency selection justification

• Provenance suitability references

• Security constraint references

• Legal constraint references

• Introduction authority references

• Continuity references

• Autonomous dependency selection

• AI-generated suitability claims

• Suitability evidence

• Evidence competence

• Evidence integrity

• Suitability validation

• Suitability uncertainty

• Suitability conflicts

• Suitability gaps

• Critical suitability gaps

• Conditional suitability

• Suitability drift

• Suitability changes

• Suitability continuity

• Suitability governance states

• Suitability recognition

• Suitability reassessment

DSM does not independently determine:

• Component identification completeness

• Complete dependency provenance

• Legitimate introduction authority

• Dependency acceptance

• Software security certification

• Software product quality

• Legal compliance

• Supplier reliability

• Long-term dependency availability

• Operational dependency continuity

• Software release readiness

• Deployment authorization

• Legal liability

DSM ends when the relevant dependency suitability conditions have been represented as a:

Governed Dependency Suitability State

with sufficient purpose, necessity, compatibility, constraint, evidence,

uncertainty, and continuity information for competent downstream SDS governance.

• Use Case 1 — An Autonomous AI Agent Selects a Popular Library That Is

Unsuitable for Production

An organization authorizes an autonomous AI development agent to build a

software service responsible for processing business transactions.

The agent identifies a popular external library that provides the required transaction-processing functionality.

The library has extensive documentation, numerous downloads, and positive

developer feedback.

The agent installs the library and integrates it into the application.

The application compiles successfully.

Functional tests pass in the development environment.

The agent concludes that the library is suitable.

However, the intended production environment has strict memory limits and

requires predictable processing latency.

The selected library consumes substantially more memory than the production

environment permits under expected workloads.

Its behavior under the required operating conditions has not been validated.

The dependency is functional.

Its production suitability has not been established.

DSM evaluates:

Software Purpose → Requirements → Dependency Necessity → Functional

Suitability → Runtime Compatibility → Resource Constraints → Suitability Evidence

The assessment identifies:

• The dependency provides the required functionality

• The agent's selection was based partly on popularity and installation

• The development tests do not establish production suitability

• Material resource requirements remain unsupported

• The selected dependency may be incompatible with the intended

convenience. operational constraints.

The relevant governance state may be:

Compatibility-Unresolved

or:

Unsuitable

depending on the available evidence and demonstrated production conditions.

The critical distinctions are:

Dependency Functionality ≠ Governed Dependency Suitability.

Development Execution Success ≠ Production Runtime Compatibility.

Resource Availability ≠ Justified Resource Consumption.

Autonomous Dependency Selection ≠ Governed Dependency Suitability.

The result: DSM prevents successful AI-generated development and passing

functional tests from being mistaken for evidence that a dependency is suitable for the environment in which the software must actually operate.

Use Case 2 — A Dependency Update Preserves Functionality but Breaks

System Compatibility

An organization operates a software platform that relies on several external libraries.

One library provides a critical data-processing function.

The existing version has been assessed as suitable for the platform's

requirements and operating environment.

A newer version becomes available.

The update includes improvements and additional functionality.

An automated dependency management process recommends the update.

The updated library installs successfully.

Its isolated functional tests pass.

However, the new version changes an interface behavior relied upon by

another component.

The individual library remains functional.

The dependency network is no longer fully compatible.

The updated software produces inconsistent results under certain operating

conditions.

DSM evaluates:

Dependency Version → Interface Compatibility → Dependency Interaction →

Operating Environment → Suitability Evidence → Suitability Drift

The assessment identifies:

• The updated library is technically installable

• The updated library performs its primary function

• A material interface relationship has changed

• The previous suitability assessment no longer fully applies

• The interaction between dependencies requires reassessment

The relevant governance state may be:

Suitability-at-Risk

or:

Unsuitable

depending on the demonstrated incompatibility and applicable requirements.

The critical distinctions are:

Latest Dependency Version ≠ Most Suitable Dependency Version.

Individual Component Compatibility ≠ Dependency Network Compatibility.

Technical Installability ≠ Demonstrated Technical Compatibility.

Previously Suitable Dependency ≠ Permanently Suitable Dependency.

The result: DSM identifies that dependency suitability belongs to the complete software context—not merely to the isolated component or its latest available version.

Canonical Closing Statement

A software dependency can be available, authentic, widely used, actively maintained, and technically functional.

None of these characteristics independently establishes that the dependency belongs in a particular software system.

A library may perform its advertised function while exceeding the resources available in production.

A framework may offer extensive capabilities while introducing unnecessary complexity.

A newer version may operate correctly in isolation while breaking compatibility with the surrounding dependency network.

An AI development agent may select a highly rated component that satisfies a coding task but fails the actual operational requirements of the software.

In every case, the component may work.

But working is not the same as being suitable.

Dependency suitability is not an inherent, universal property of a software component.

It is a governed relationship between the component and the specific purpose, requirements, environment, constraints, and conditions under which the software relies upon it.

Dependency Functionality ≠ Governed Dependency Suitability.

Dependency Convenience ≠ Dependency Necessity.

Technical Installability ≠ Demonstrated Technical Compatibility.

Individual Component Compatibility ≠ Dependency Network Compatibility.

Previously Suitable Dependency ≠ Permanently Suitable Dependency.

DSM establishes the methodological requirement that software dependencies must be evaluated in the context of the systems that use them.

This requirement remains unchanged whether the dependency is selected by a human developer, recommended by an AI assistant, introduced by an autonomous agent, or updated through an automated process.

The selection mechanism may change.

The component may change.

The software environment may change.

The requirement to establish contextual suitability remains.

DSM does not confuse a justified dependency choice with legitimate authorization to introduce it.

It does not confuse technical compatibility with security acceptance.

And it does not confuse present suitability with guaranteed operational continuity.

Instead, it establishes the evidence-supported conditions under which

dependency reliance can be considered appropriate for its intended software context.

DSM transforms dependency selection from a question of whether a component

can perform a function into a governed determination of whether that component should be relied upon for a particular software purpose, under particular conditions, with sufficient evidence that the reliance is justified and compatible.

A dependency is not suitable merely because it works. It is suitable when its use is justified by the software's requirements, supported by evidence, compatible with its operating conditions, and bounded by the constraints under which that suitability remains valid.